

PRD 3.4 — Production Order & Task

Product Requirements Document · portfolio sample. Company, client, and product names have been genericized; structure and logic are unchanged.

- **Status:** Draft v1 — the Production Order & Task contract, extracted as its own PRD (05 Aug renumbering): the entity PRD 3.2 creates at submit, PRD 3.1 records against, PRD 3.6 renders to the crew, and PRD 3.5 reads progress from. Fills the "Production Order / KDS contract" both planning PRDs cited but no doc owned
- **Owner:** Jauhar Nurandyo
- **Epic:** Project Brief - EPIC 3 - Production Planning
- **Phase:** P1 · the execution spine — between the plan and the kitchen
- **Tenant scope:** the client — QSR chain
- **Model:** exit-only deduction unchanged — the Order and its tasks move **no stock**; the stock-in happens at 3.1's confirmation, deduction at the exits (EPIC 2). The Order is a work container, never a ledger document
- **Surface:** none of its own — the Order/Task renders through consumers: the SM's plan detail (3.2), the reports (3.5), and the crew KDS (3.6)

1. Objective

Define the **Production Order and Production Task** — the objects that carry a submitted plan into the kitchen — once, in one place. Both planning flows create them (revenue-based 3.2 now; template-based 3.7 when it returns); the actual entry (3.1) confirms against them; the KDS (3.6) renders them; the reports (3.5) read their progress. Until this PRD, every one of those docs pointed at a "Production Order / KDS contract" that existed nowhere. This is that contract.

One Production Order per plan; one task per FG/SFG × slot. The Order is the folder; tasks are the work. Nothing here posts, reserves, or gates stock.

Scope boundary. This PRD owns the Order/Task **entities, their state machines, the creation contract, the dispatch/feed contract, and the progress rollup**. It does **not** own: what a confirmation captures or the stock-in it posts (3.1), the KDS's form and interaction (3.6), the plan that generates the quantities (3.2 / 3.7), or reporting surfaces (3.5). **Ownership transfer note:** PRD 3.1 previously carried the task lifecycle in prose (its §5.1 / §5.3); that ownership moves here. 3.1 keeps capture, production_entry, and the stock-in — a pointer edit to 3.1 is queued (§9).

2. Problem Statements

- Five PRDs reference the Production Order; none specs it. Every doc says "owned by the Production Order contract" — an empty chair. Dev cannot build an entity whose spec is distributed across pointers.
- The plan is a paper act and the stock-in is an entry-level event — between them, the kitchen needs a **work object**: what to cook, when it becomes current, what is done. Without it, the KDS has nothing to render and progress has nothing to roll up.
- The same FG can be needed by many menus in the same hour; without a single task grain, the kitchen gets duplicate tickets (the aggregation invariant, 3.2 §4, must land as *one task*).
- The order-grain question (per-plan vs per-slot) stayed open across three docs; it must close here, once, for all consumers.

3. Goals

- **One entity spec:** Order (one per plan) + Task (one per FG/SFG × slot, from the plan's aggregated cook list), with their state machines.
- **A creation contract** any planning flow satisfies: submit → one Order + its tasks, snapshot-backed, zero-aggregate slots producing no task.
- **A dispatch/feed contract** the KDS consumes: clock-driven release (Pending 1h before slot start), current-slot activation, station routing.
- **A progress contract** the tableview and reports consume: task states roll up to one Order progress; no per-slot order statuses exist.
- Stock-free by construction: no movement, no reservation, no gate, at any state transition.

4. The Model

4.1 The two objects

Object	Grain	Carries	State
Production Order	One per submitted plan (branch × date)	Plan linkage, dispatch config, rollup progress	Derived: OPEN → IN_PROGRESS → COMPLETED (rollup of its tasks; never set directly)
Production Task	One per FG/SFG × slot (MVP slot = hour; Daypart windows when 3.7 returns)	product_variant_id, slot, planned qty (the target, from the aggregated cook list), station	PENDING → IN_PROGRESS → DONE

The task inherits the aggregation invariant: its planned qty is the cook-list aggregate — the sum across every menu needing that FG in that slot. Menus never appear on a task. Mashed Potato at 10:00 is one task, whatever sold it.

The Order is a container, not a step. It has no approval, no release action, no per-slot breakdown of its own status. Its state is purely derived: OPEN (no task started) → IN_PROGRESS (any task active or done) → COMPLETED (all tasks terminal). Progress = done-tasks / total, plus a qty-weighted % for display.

Retired with per-slot orders: the In queue → Cooking → Ready per-slot order statuses (3.7 §5.4's working assumption). Under one-order-per-plan those semantics move to the **task**: In queue ≈ Pending, Cooking ≈ In Progress, Ready ≈ Done. No consumer may depend on a per-slot order status (§9).

4.2 The task state machine

- **PENDING** — the task exists (created at submit) but is not yet on the floor. It **releases** — becomes visible on the KDS feed — exactly **1 hour before its slot start** (clock-driven; a 15:00 task surfaces at 14:00). Before release it is visible in the Order detail (SM view), not the kitchen feed.
- **IN_PROGRESS** — the slot's timeframe is current. The task is cookable and counts toward the Order's progress denominator display. **Confirmation is available from release onward** — cooked-ahead is legal (confirm before the slot elapses).
- **DONE** — a confirmation (3.1) covering the task moves it here. The confirmation carries the **actual** (plan pre-filled, editable — 3.1's model); multiple batch entries against one task each post their own stock-in, and the task is Done when the operator closes it. A voided-entry task (3.1 §5.5) reverts to its time-appropriate state.

- **No task is ever blocked** — no stock precondition, no approval, at any transition. A slot passing without confirmation leaves the task IN_PROGRESS (visibly overdue); end-of-day unconfirmed tasks surface in the Order detail as unworked, never auto-confirmed. **Nothing is ever assumed cooked.**

4.3 Dispatch — the feed contract (what 3.6 consumes)

The Order exposes a per-station, time-ordered feed of released tasks: task_id, product_variant, slot, planned_qty, station, state. Release timing (the 1h rule) and state transitions are computed here, server-side — **the KDS renders states; it never computes them.** How tickets look, fast-Bump, and offline queue-and-replay are 3.6's; that offline replay must not double-advance a task (the entry idempotency in 3.1 §5.4 covers the movement; the task transition is idempotent on the entry id here).

Station routing: each task routes to exactly one station via the FG's station_id. Templateless orphaned the template's station field — **interim home: the FG product record** (cross-PM, §9). Until the master resolves, all tasks fall back to a single undifferentiated feed. The contract depends on station_id; V1 tolerates its absence.

4.4 What the Order never does

No stock movement at creation, transition, or completion. No reservation. No gate on sales, on 3.1 entries, or on the next day's plan. COMPLETED is informational — EPIC 4's future REWORK / DISPOSE job lines attach to the Order at close but are not rendered before EPIC 4 ships.

5. System Flow

5.1 Creation — the contract planning flows satisfy

On plan submit, atomically with the plan's lock + snapshot: create **one Order** linked to the plan; create **one task per cook-list line with aggregate > 0** (zero-aggregate slots produce no task); stamp each task with variant, slot, planned qty, station. The Order's lines are the snapshot's cook list — later menu / BOM edits never reach a created Order. There is no partial creation: submit fails whole or succeeds whole.

5.2 The day

Tasks release on the clock (the 1h rule) → activate with their slot → confirm via 3.1 (SM form Phase 1; KDS Phase 2 through 3.6). The Order's rollup updates on every task transition; the tableview (3.2 Story 6) and Monitoring (3.5) read it live.

5.3 Corrections

A same-day entry edit adjusts the posted actual (3.1); the task stays Done. A void reverts the task to its time-appropriate state and re-opens the Order if it was Completed. Plan-level corrections do not exist — a locked plan's Order is never regenerated.

6. Data Model

Entity	Notes
production_order	One per plan: id, plan_id (unique), branch_id, planning_date, derived status (OPEN / IN_PROGRESS / COMPLETED), progress (computed), audit set
production_task	id, order_id, product_variant_id (FG / SFG), slot (hour; window-capable), planned_qty, station_id (nullable, V1 fallback), state (PENDING / IN_PROGRESS / DONE), released_at, done_at, audit set

Entity	Notes
Links	3.1's production_entry.production_task_id → here; 3.2's production_plan_cook_line is the creation source

No movement tables. State transitions are audited (who / what / when — clock, entry id, or void id).

7. User Stories & Acceptance Criteria

Story 1 — Submit creates one Order and its tasks (the creation contract)

Field	Details
User Story	As the system, when a plan is submitted I create exactly one Production Order and one task per FG/SFG × slot from its aggregated cook list, so the kitchen receives one work object per day with no duplicate tickets.
Priority	P0
BRD	— (structural; implements 3.2 §5.4)

Acceptance Criteria

- Submit creates **exactly one** Order per plan, atomically with the plan's lock + snapshot; no partial creation
- Tasks are created one per cook-list line with **aggregate > 0**; a zero-aggregate FG × slot produces no task; no task references a menu
- Each task carries variant, slot, planned qty (the aggregate), and station (nullable in V1)
- Later menu / price / BOM edits never alter a created Order or its tasks (snapshot-backed)
- Creation writes no stock movement and performs no stock check

Scenario: One Order, aggregated tasks

Given a submitted plan whose cook list holds Mashed Potato 10:00 = 15 (from three menus)

When the Order is created

Then exactly one Order exists for the plan

And exactly one Mashed Potato 10:00 task exists, planned 15

And no task names any menu

Scenario: Zero aggregate, no task

Given the cook list's Spicy Chicken 21:00 aggregate is 0

When tasks are created

Then no Spicy Chicken 21:00 task exists

Story 2 — Tasks release on the clock and move through their lifecycle

Field	Details
User Story	As the outlet, the day's tasks appear as their time approaches and move Pending → In Progress → Done as they are worked, so what to cook next is always current without anyone dispatching by hand.
Priority	P0
BRD	— (carries 3.1's former §5.3, ownership moved here)

Acceptance Criteria

- A task **releases** exactly 1 hour before its slot start (clock-driven, server-side); before release it is absent from the kitchen feed but visible in the Order detail
- A task turns IN_PROGRESS when its slot becomes current; **confirmation is available from release onward** (cooked-ahead is legal)
- A confirmation (3.1) moves the task to DONE; a void reverts it to its time-appropriate state
- A slot passing unconfirmed leaves the task IN_PROGRESS and visibly overdue; **nothing is auto-confirmed**
- No transition is stock-gated or approval-gated; state transitions are idempotent on their trigger (a replayed confirmation does not double-advance)

Scenario: Clock-driven lifecycle
 Given a Crispy Chicken 15:00 task
 When the clock reaches 14:00
 Then the task releases as PENDING on the feed
 When the clock reaches 15:00
 Then it turns IN_PROGRESS
 When its confirmation posts (3.1)
 Then it is DONE and the Order's progress advances

Scenario: Overdue is visible, never assumed
 Given the 12:00 slot has passed with no confirmation
 Then the task remains IN_PROGRESS, flagged overdue
 And no actual and no stock-in exist for it

Story 3 — The Order's status is a rollup, never a step

Field	Details
User Story	As the SM, my plan's Order shows one honest progress figure rolled up from its tasks, so I track the day at a glance — and nobody manages order states by hand.
Priority	P0
BRD	PP-06.2 (progress on the plan list)

Acceptance Criteria

- Order status is **derived**: OPEN (no task started) → IN_PROGRESS (≥1 task active / done) → COMPLETED (all tasks terminal); it is never directly settable
- Progress exposes done / total tasks and a qty-weighted %; recomputed on every task transition
- A void on a Done task re-opens a Completed Order
- **No per-slot order status exists**; slot-level visibility is the task list, grouped by slot in the Order detail (the granularity survives; the per-slot *order* does not)
- The tableview (3.2 Story 6) and Production Monitoring (3.5) read this rollup; they never compute their own

Scenario: Rollup, and reopen on void
 Given an Order whose 24 tasks are all DONE (COMPLETED)
 When one task's entry is voided
 Then that task reverts and the Order returns to IN_PROGRESS

Story 4 — The KDS feed contract (what 3.6 renders)

Field	Details
User Story	As the crew KDS (3.6), I consume a per-station, time-ordered feed of released tasks with their states computed server-side, so the screen renders work without owning any lifecycle logic.
Priority	P1 — the contract is P0 to <i>define</i> ; its consumer ships Phase 2
BRD	— (the seam 3.6 builds on)

Acceptance Criteria

- The feed exposes, per station: released tasks with task_id, variant, slot, planned qty, state — ordered by slot
- Release timing and all state transitions are computed here; the KDS renders, never computes
- Station routing keys on the FG's station_id; a null station falls back to the single undifferentiated feed (V1)
- A replayed / offline-queued confirmation (3.6) advances the task **once** — transition idempotent on the entry id
- The feed carries no stock figures — execute-first, per 3.1's surface rule

Scenario: Station feed

Given Fryer-station tasks at 14:00 and 15:00 are released and a Prep task is not yet released

When the Fryer KDS reads its feed

Then it receives the two Fryer tasks in slot order and nothing else

Story 5 — Both planning flows create through one contract

Field	Details
User Story	As the platform, the revenue flow today and the template flow when it returns create Orders and tasks through the identical contract, so the kitchen and every consumer are indifferent to how the plan was made.
Priority	P1
BRD	— (future-proofing; 3.7's re-entry)

Acceptance Criteria

- The creation contract (Story 1) takes an aggregated cook list — variant × slot × qty — regardless of its origin (3.2's menu seeding; 3.7's template autofill)
- The slot type admits hours (MVP) and Daypart windows (3.7's return) without a task-model change
- No field on Order or task references menus, templates, or revenue — planning-basis-free by construction

Scenario: Basis-blind kitchen

Given an Order created by the revenue flow and (future) one by the template flow

When the crew work their tasks

Then the task shape, lifecycle, and feed are identical in both

8. Dependencies

Dependency	Direction	Note
PRD 3.2 (and 3.7 on return)	upstream	Creates through the Story-1 contract; supplies the aggregated cook list

Dependency	Direction	Note
PRD 3.1 Actual Production	sibling	Confirmations / voids drive task transitions; owns the entry, the actual, and the stock-in
PRD 3.6 Crew KDS	downstream	Renders the Story-4 feed; owns interaction + offline mechanics
PRD 3.5 Reports	downstream	Reads the rollup and task states
station_id on the FG product record	upstream, cross-PM	Templateless orphaned the station; interim home is the FG; single-feed fallback until the master lands
EPIC 4	downstream	REWORK / DISPOSE job lines attach at Order close, post-EPIC-4

9. Risks & Decisions

Item	Type	Severity	Note
Order grain — one per plan	Decided (PM, 04 Aug)	—	Closes the question open across 3.1 / 3.5 / 3.7. Tasks carry slot granularity; the per-slot <i>order</i> and its In queue → Cooking → Ready statuses are retired — no consumer may depend on them
Ownership transfer from 3.1	Decided (05 Aug)	Low	Task lifecycle + release rule move here from 3.1 §5.1 / §5.3; 3.1 keeps capture / entry / stock-in. Queued edit: 3.1 gets pointer lines to 3.4 (its prose still carries template-era lifecycle text)
Slot type spans hour + window	Decided	—	MVP = hour; window-capable for 3.7's return; no remodel later
Station on the FG record	! Provisional — cross-PM	Medium	Interim home after templateless; single-feed fallback until the station master resolves. Owner: station-master PM
Overdue semantics	! Provisional	Low	Slot-passed-unconfirmed = IN_PROGRESS + overdue flag, never auto-anything. Alternative (auto-expire state) rejected as an assumed actual
Void reverts task state	Decided	Low	Mirrors 3.1 §5.5; reopens a Completed Order
3.7's per-slot prose	Follow-up	Low	3.7 (parked) still describes per-slot orders; the parking note should point here rather than rewriting a dormant doc

This PRD is the contract layer: 3.2 / 3.7 create through it, 3.1 confirms against it, 3.6 renders it, 3.5 reads it. It moves no stock and owns no screen.